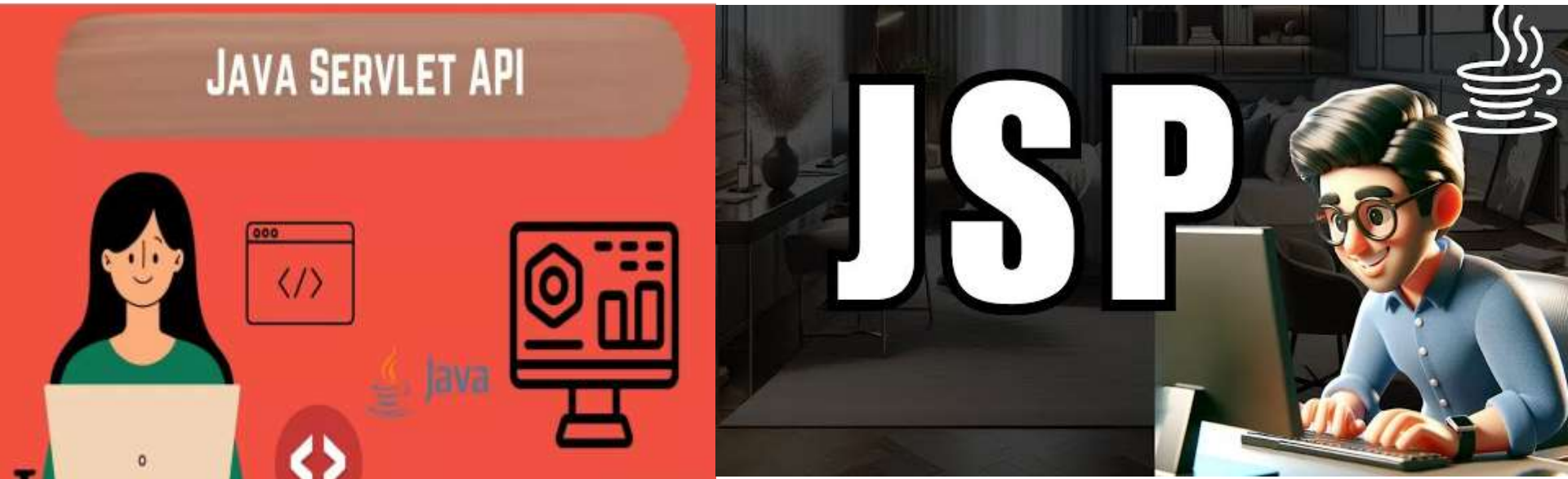


Unit 7: Servlets and Java Server pages (8 Hrs.)

PART-1



By: Kabita Dhital

7th sem(CSIT)

Content

7.1. Web Container, Introduction to Servlets, Life cycle of servlets, The servlet APIs, Writing Servlet Programs, Reading Form Parameters, Processing Forms, Handling HTTP Request and Response (GET / POST Request), Database Access with Servlets, Handling Cookies and Session.

7.2. Servlet vs JSP, JSP Access Model, JSP Syntax (Directions, Declarations, Expression, Scriptlets, Comments), JSP Implicit Objects, Object Scope, Processing Forms, Database Access with JSP.

7.3. Introduction to Java Web Frameworks

Steps

- 1)Download apache Tomcat
- 2)Using web-app artifact from maven
- 3)API(Jakarta)-**Jakarta is an API**, or more precisely a set of **standardized Java APIs**, under the **Jakarta EE (Enterprise Edition)** umbrella, designed for building web and enterprise-level applications.

Jakarta Servlet for HTTP request/response handling.

Jakarta Server Pages (JSP) for view rendering

Jakarta EE is the evolution of **Java EE**, now managed by the **Eclipse Foundation**

Jakarta APIs **don't run by themselves** — they need a **server** (like Tomcat, WildFly, or Payara) to provide the actual runtime implementation.

Apache Tomcat
Version

Servlet API

Package Prefix

Tomcat 9.0.107

4.0

javax.servlet.*

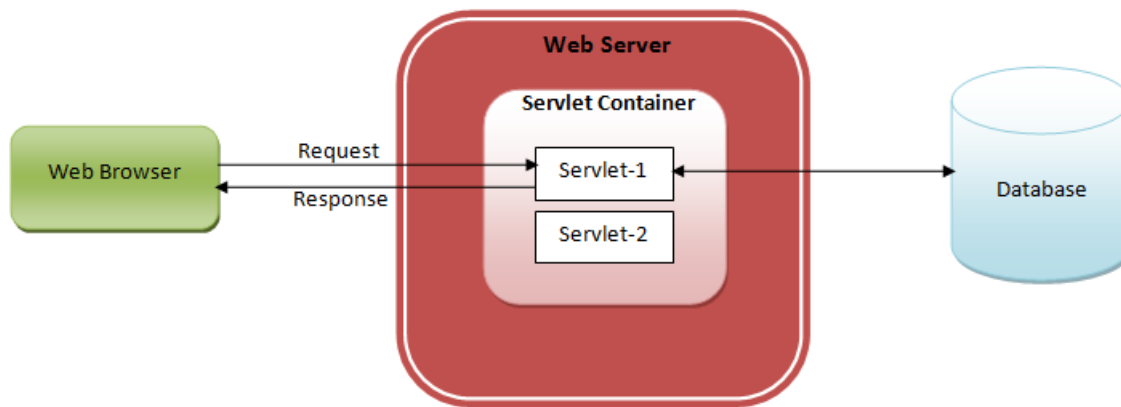
Tomcat 10.1.20

6.0

jakarta.servlet.*

Web Container

- A web container in Java, also known as a servlet container, manages the lifecycle of web components like servlets and JSPs, handling client requests and responses. It acts as an intermediary between the web server and these components, providing a runtime environment for them to execute.
- The web client requests a URL to the web server.
- The web container in the server maps it to the requested servlet.
- The servlet then loads the methods and functions.
- The servlet processes the response to the container, which finally reaches the client through the server.

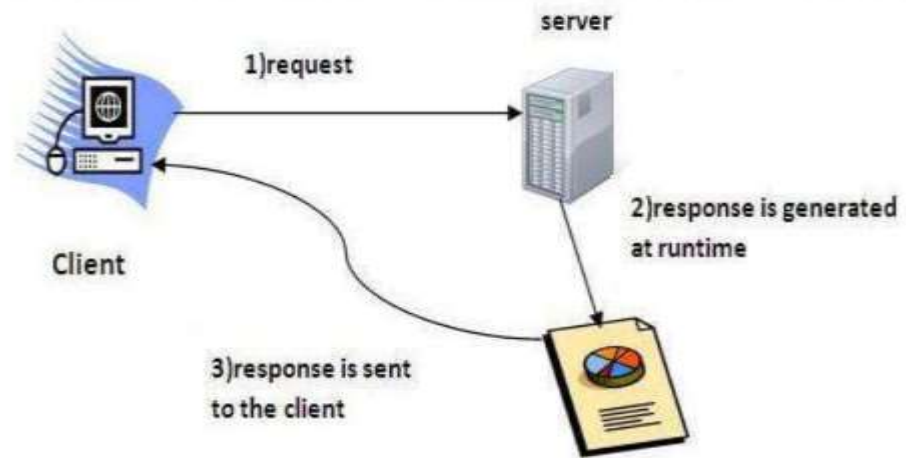


Web Terminology

Servlet Terminology	Description
Website: static vs dynamic	It is a collection of related web pages that may contain text, images, audio and video.
HTTP	It is the data communication protocol used to establish communication between client and server.
HTTP Requests	It is the request send by the computer to a web server that contains all sorts of potentially interesting information.
Get vs Post	It gives the difference between GET and POST request.
Container	It is used in java for dynamically generating the web pages on the server side.
Server: Web vs Application	It is used to manage the network resources and for running the program or software that provides services.
Content Type	It is HTTP header that provides the description about what are you sending to the browser.

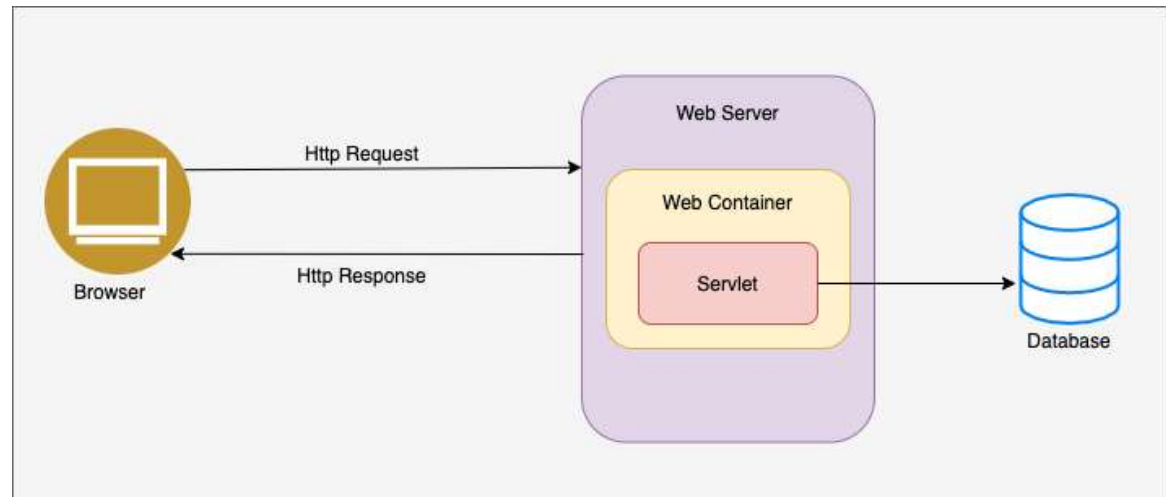
What is Servlets?

- **Servlet** technology is used to create a web application (resides at server side and generates a dynamic web page).
- Servlet is a web component that is deployed on the server to create a dynamic web page.
- **Properties of Servlets :**
 - Servlets work on the server-side.
 - Servlets are capable of handling complex requests obtained from web server.



Introduction to Servlets

- Servlets are small programs that execute on there server side of a web connection. Just as applets dynamically extend the functionality of a Web browser, servlets dynamically extend the functionality of a Web server.
- A servlet is a Java programming language class used to extend the capabilities of servers that host applications accessed via a request-response programming model. Although servlets can respond to any type of request, they are commonly used to extend the applications hosted by Web servers. For such applications, Java Servlet technology defines HTTP-specific servlet classes.
- The `javax.servlet` and `javax.servlet.http` packages provide interfaces and classes for writing servlets. All servlets must implement the Servlet interface, which defines life-cycle methods.



Life cycle of servlets

- Three methods are central to the life cycle of a servlet. These are `init( )`, `service( )`, and `destroy( )`.
- They are implemented by every servlet and are invoked at specific times by the server.
- Let us consider a typical user scenario to understand when these methods are called. First, when a user enters a Uniform Resource Locator (URL) to a Web browser.
- The browser then generates an HTTP request for this URL. This request is then sent to the appropriate server.
- Second, this HTTP request is received by the Web server. The server maps this request to a particular servlet. The servlet is dynamically retrieved and loaded into the address space of the server.
- Third, the server invokes the `init( )` method of the servlet. This method is invoked only when the servlet is first loaded into memory. It is possible to pass initialization parameters to the servlet so it may configure itself.
- Fourth, the server invokes the `service( )` method of the servlet. This method is called to process the HTTP request. It is possible for the servlet to read data that has been provided in the HTTP request. It may also formulate an HTTP response for the client. The servlet remains in the server's address space and is available to process any other HTTP requests received from clients. The `service( )` method is called for each HTTP request.

- Finally, the server may decide to unload the servlet from its memory. The server calls the `destroy()` method to relinquish any resources such as file handles that are allocated for the servlet. Important data may be saved to a persistent store. The memory allocated for the servlet and its objects can then be garbage collected.

State of sevlet life cycle

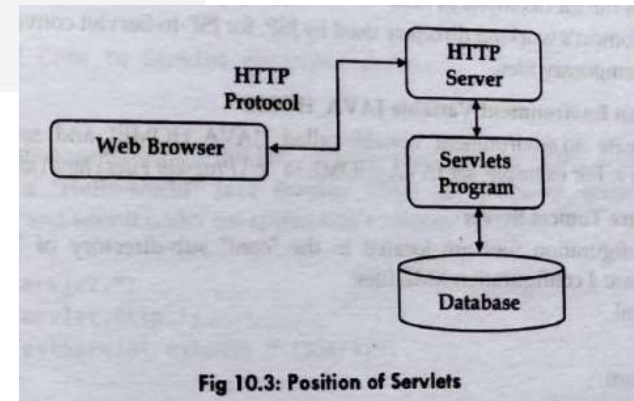
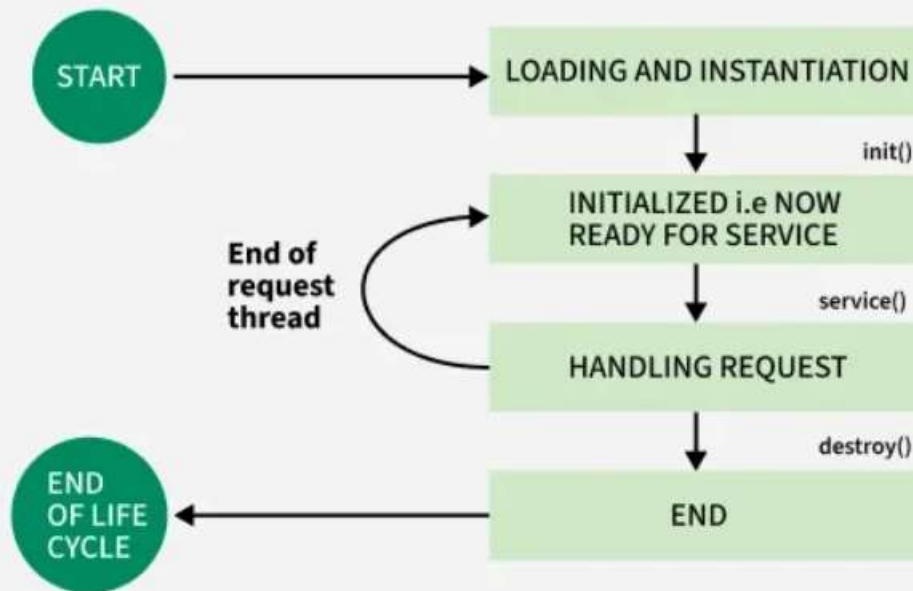


Fig 10.3: Position of Servlets

Life Cycle of a Servlet

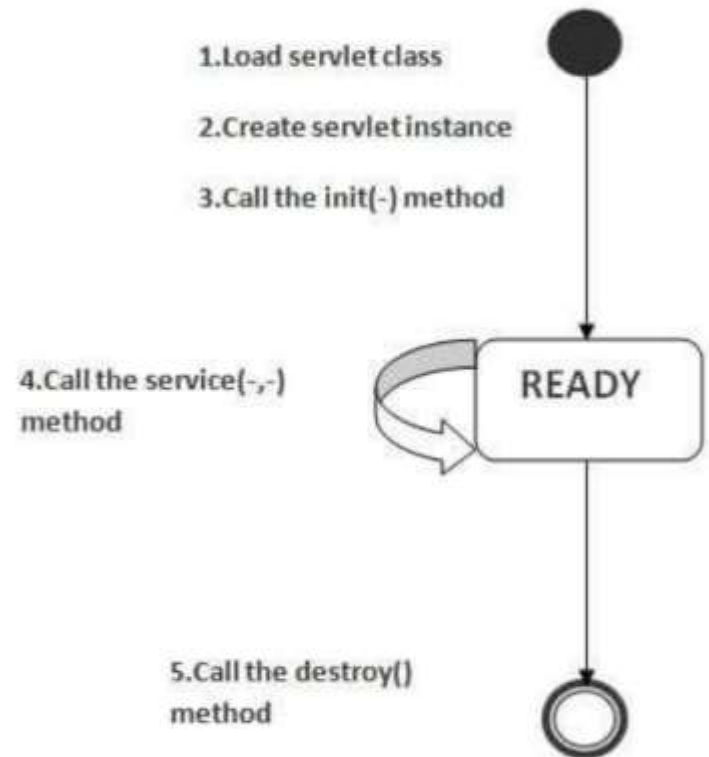
Life cycle methods of a Servlet



Called only once

service() handles multiple client requests and send sends response

Called only once



1. init() method:

- The **Servlet.init()** method is called by the Servlet container to indicate that this Servlet instance is instantiated successfully and is about to put into service.

```
public class MyServlet implements Servlet{  
    public void init(ServletConfig config) throws ServletException {  
        //initialization code  
    }  
    //rest of code  
}
```

2. service() method:

The service() method of the Servlet is invoked to inform the Servlet about the client requests. This method uses ServletRequest object to collect the data requested by the client. This method uses ServletResponse object to generate the output content.

```
public class MyServlet implements Servlet{  
    public void service(ServletRequest res, ServletResponse res)  
        throws ServletException, IOException {  
        // request handling code  
    }  
    // rest of code  
}
```

3. destroy() method:

- The destroy() method runs only once during the lifetime of a Servlet and signals the end of the Servlet instance.
- As soon as the destroy() method is activated, the Servlet container releases the Servlet instance.

```
public void destroy()
```

The servlet APIs

- Two packages contain the classes and interfaces that are required to build servlets. These are `javax.servlet` and `javax.servlet.http`. They constitute the Servlet API. These packages are not part of the Java core packages. Instead, they are standard extensions. Therefore, they are not included in the Java Software Development Kit.
- You must download Tomcat or Glass Fish server to obtain their functionality.
- The `javax.servlet` package contains a number of interfaces and classes that establish the framework in which servlets operate.
- The most significant of these is `Servlet`. All servlets must implement this interface or extend a class that implements the interface.
- The `ServletRequest` and `ServletResponse` interfaces are also very important.

Interface	Description
Servlet	Declares life cycle methods for a servlet.
ServletConfig	Allows servlets to get initialization parameters.
ServletContext	Enables servlets to log events and access information about their environment.
ServletRequest	Used to read data from a client request.
ServletResponse	Used to write data to a client response.
SingleThreadModel	Indicates that the servlet is thread safe.

The following table summarizes the **core classes** that are provided in the javax.servlet package.

Class	Description
GenericServlet	Implements the Servlet and ServletConfig interfaces.
ServletInputStream	Provides an input stream for reading requests from a client.
ServletOutputStream	Provides an output stream for writing responses to a client.
ServletException	Indicates a servlet error occurred.
UnavailableException	Indicates a servlet is unavailable.

Servlet Interface

- All servlets must implement the Servlet interface. It declares the `init( )`, `service( )`, and `destroy( )` methods that are called by the server during the life cycle of a servlet.
- The `ServletRequest` interface is implemented by the server. It enables a servlet to obtain information about a client request.
- The `ServletResponse` interface is implemented by the server. It enables a servlet to formulate a response for a client
- The methods defined by Servlet are shown below:

Method	Description
<code>void destroy()</code>	Called when the servlet is unloaded.
<code>ServletConfig getServletConfig()</code>	Returns a ServletConfig object that contains any initialization parameters.
<code>String getServletInfo()</code>	Returns a string describing the servlet.
<code>void init(ServletConfig sc)</code> throws <code>ServletException</code>	Called when the servlet is initialized. Initialization parameters for the servlet can be obtained from <code>sc</code> . An UnavailableException should be thrown if the servlet cannot be initialized.
<code>void service(ServletRequest req,</code> <code>ServletResponse res)</code> throws <code>ServletException,</code> <code>IOException</code>	Called to process a request from a client. The request from the client can be read from <code>req</code> . The response to the client can be written to <code>res</code> . An exception is generated if a servlet or IO problem occurs.

JSP - Servlets: A servlet example

- ```
<!DOCTYPE html>
<html>
<head>
 <title>Registration Form</title>
 <link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
 <h2>User Registration</h2>
 <form action="loginMsg" method="post">
 <label>Full Name:</label>

 <input type="text" name="fullname" required>

 <label>Email:</label>

 <input type="email" name="email" required>

 <label>Username:</label>

 <input type="text" name="username" required>

 <label>Password:</label>

 <input type="password" name="password" required>

 <input type="submit" value="Register">
 </form>
</body>
</html>
```

- package servlets;

```
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
import java.io.PrintWriter;
```

```
public class loginMsg extends HttpServlet {
```

```
 @Override
```

```
 public void service(HttpServletRequest req, HttpServletResponse res) throws IOException {
```

```
 // Set the content type so browser treats it as HTML
 //res.setContentType("text/html");
```

```
 // Get the PrintWriter to write the response
 PrintWriter out = res.getWriter();
```

```
 // Fetch form data from request
```

```
 String fullname = req.getParameter("fullname");
 String email = req.getParameter("email");
 String username = req.getParameter("username");
 String password = req.getParameter("password");
```

```
 // Generate HTML response
```

```
 out.println("<html><body>");
 out.println("<h2>Registration Successful!</h2>");
 out.println("<p>Full Name: " + fullname + "</p>");
 out.println("<p>Email: " + email + "</p>");
 out.println("<p>Username: " + username + "</p>");
 // Don't display password in real applications
 out.println("</body></html>");
```

```
 }
```

```
}
```

- **Define your servlet in "web.xml"**
- `<!DOCTYPE web-app PUBLIC  
"-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"  
"http://java.sun.com/dtd/web-app_2_3.dtd" >`

```
<web-app>
 <display-name>Archetype Created Web Application</display-
name>
```

```
 <servlet>
 <servlet-name>loginMsg</servlet-name>
 <servlet-class>servlets.loginMsg</servlet-class>
 </servlet>
```

```
 <servlet-mapping>
 <servlet-name>loginMsg</servlet-name>
 <url-pattern>/loginMsg</url-pattern>
 </servlet-mapping>
```

```
</web-app>
```

# Why web.xml is used in servlets?

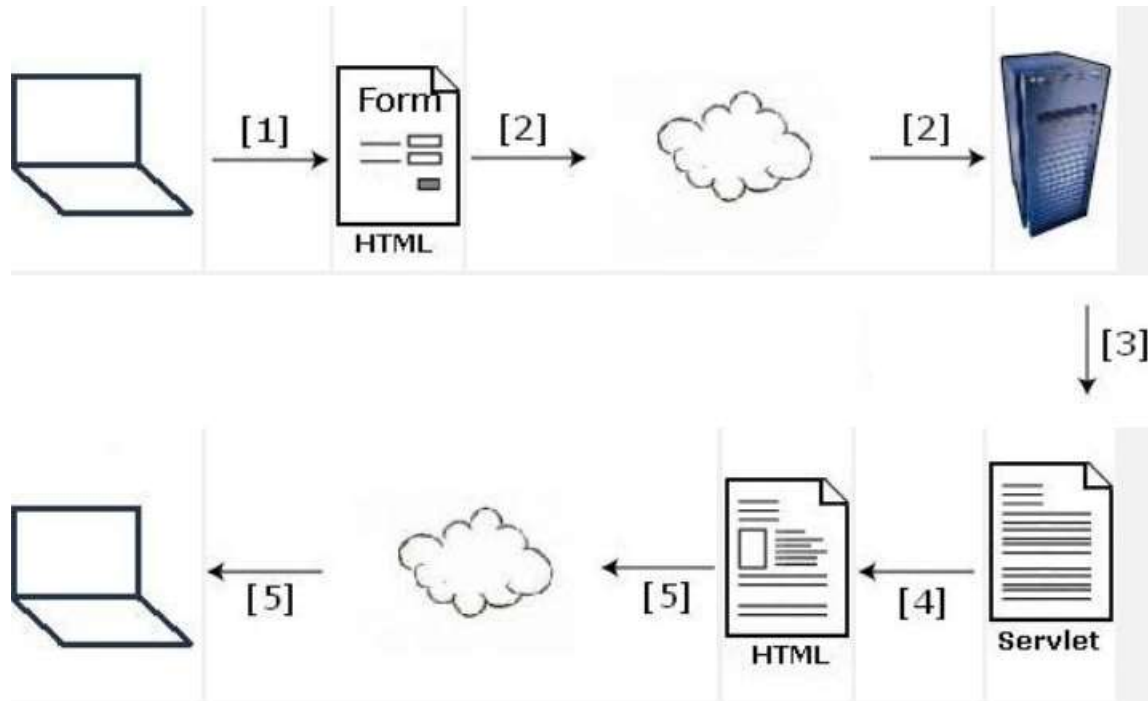
---

- **web.xml** defines mappings between URL paths and the **servlets** that handle requests with those paths. The **web** server uses this configuration to identify the **servlet** to handle a given request and call the class method that corresponds to the request method (e.g. the `doGet()` method for HTTP GET requests).

- **Add your project to Tomcat**

**Start "Tomcat":** In the "Servers" view, R-click Tomcat-server record / Start

- **Restart the server** and test your project again using `http://localhost/yourProjectName` - in our example, the URL will be <http://localhost/ServletExample/>.
- Note that the server does not recognize changes in "web.xml" except if you restarted, or started, the server.



# Home Work

- Write a Steps to write Java Servlet Program.
- Compare and contrast between Servlets and Java Server pages? How web server responds to servlets and JSP page? Write down sample example. Handling HTTP Request and Response using servlets.
- Discuss different methods used in life cycle of servelet.

# How to write Servlet hello world Example

Servlets are Java classes which service HTTP requests and implement the **javax.servlet.Servlet** interface. Web application developers typically write servlets that extend `javax.servlet.http.HttpServlet`, an abstract class that implements the Servlet interface and is specially designed to handle HTTP requests.

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloWorld extends HttpServlet {

 private String message;

 public void init() throws ServletException {
 // Do required initialization
 message = "Hello World";
 }
}
```

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException {

 // Set response content type
 response.setContentType("text/html");

 // Actual logic goes here.
 PrintWriter out = response.getWriter();
 out.println("<h1>" + message + "</h1>");
}

public void destroy() {
 // do nothing.
}
```

## Compiling a Servlet

```
$ javac HelloWorld.java
```

If everything goes fine, above compilation would produce **HelloWorld.class** file in the same directory.

---

## Servlet Deployment

**web.xml**

```
<servlet>
 <servlet-name>HelloWorld</servlet-name>
 <servlet-class>HelloWorld</servlet-class>
</servlet>

<servlet-mapping>
 <servlet-name>HelloWorld</servlet-name>
 <url-pattern>/HelloWorld</url-pattern>
</servlet-mapping>
```



# Reading Form Parameters in Servlet

- Servlets parse the form(client) data automatically using the following methods depending on the situation –
  - getParameter()** – You call request.getParameter() method to get the value of a form parameter.
  - getParameterValues()** – Call this method if the parameter appears more than once and returns multiple values, for example checkbox.
  - getParameterNames()** – Call this method if you want a complete list of all parameters in the current request.
- Reading Servlet parameters, allows your server-side Servlet to access information typically submitted through HTML forms, URL query strings, or other client requests.
- When a web browser sends an HTTP request to a Servlet, it often includes various pieces of data, known as parameters. These parameters are crucial for passing information from the client (e.g., a user filling out a login form, selecting an option from a dropdown, or clicking a link with query string data) to the server-side Servlet for processing. Understanding how to correctly perform **Reading Servlet parameters** is thus paramount for any interactive web application.

- The Servlet API provides robust methods within the `HttpServletRequest` object to facilitate this parameter retrieval. Typically, parameters are sent using either the HTTP GET or POST methods.

### 1) `getParameter(String name)`

- **Purpose:** This is the most frequently used method for reading a single parameter value. It retrieves the value of a specified request parameter as a `String`.
- **Behavior:** If the parameter exists with multiple values (e.g., multiple checkboxes with the same name), it returns only the *first* value. If the parameter does not exist, it returns `null`.
- **Relevance:** This method is ideal when you expect only one value for a given parameter, such as a text input, password, or radio button selection.

```
// Assuming a form field like <input type="text" name="username">
String username = request.getParameter("username");
if (username != null) {
 // Process the username
 System.out.println("Username: " + username);
} else {
 System.out.println("Username parameter not found.");
}
```

## 2) **getParameterValues(String name)**

- **Purpose:** This method is specifically designed for parameters that might have multiple values. It returns an array of String objects containing all values for the specified request parameter.
- **Behavior:** If the parameter exists and has multiple values (e.g., a group of checkboxes with the same name), it returns all of them in a String array. If the parameter does not exist, it returns null.
- **Relevance:** This method is crucial when dealing with multi-select list boxes or groups of checkboxes where a user can select more than one option.

```
String[] hobbies = request.getParameterValues("hobbies");
if (hobbies != null) {
 for (String hobby : hobbies) {
 System.out.println("Hobby: " + hobby);
 }
} else {
 System.out.println("Hobbies parameter not found.");
}
```

### 3) **getParameterNames()**

- **Purpose:** This method provides an Enumeration of String objects, each representing the name of a parameter contained in the client request.
- **Behavior:** It allows you to iterate through all parameter names present in the request without knowing them beforehand.
- **Relevance:** This is particularly useful for debugging or when dealing with dynamic forms where parameter names might not be fixed.
- **Note:** lets you **dynamically access all form inputs**. Useful when the form fields are not fixed or when debugging.

```
// Getting all parameter names
Enumeration<String> paramNames = request.getParameterNames();
while (paramNames.hasMoreElements()) {
 String paramName = paramNames.nextElement();
 System.out.println("Parameter Name: " + paramName);
 // You can then use getParameter() or getParameterValues()
 // to retrieve the value(s) for this paramName
 String[] paramValues = request.getParameterValues(paramName);
 for (String val : paramValues) {
 System.out.println(" Value: " + val);
 }
}
```

# HTTP Methods

- The Hypertext Transfer Protocol (HTTP) is designed to enable communications between clients and servers.
- HTTP works as a request-response protocol between a client and server.
- Example: A client (browser) sends an HTTP request to the server; then the server returns a response to the client. The response contains status information about the request and may also contain the requested content.
- GET is used to request data from a specified resource.
- POST is used to send data to a server to create/update a resource.

# Handling HTTP Request and Response (GET / POST Request)

- To handle HTTP requests in a servlet, extend the `HttpServlet` class and override the servlet methods that handle the HTTP requests that your servlet supports. This lesson illustrates the handling of GET and POST requests. The methods that handle these requests are `doGet` and `doPost`.
- Handling GET requests involves overriding the `doGet` method. The following example shows the `BookDetailServlet` doing this.

```
public class BookDetailServlet extends HttpServlet {
 public void doGet (HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException
 {
 ...
 // set content-type header before accessing the Writer
 response.setContentType("text/html");
 PrintWriter out = response.getWriter();

 // then write the response
 out.println("<html>" +
 "<head><title>Book Description</title></head>" +
 "...");

 //Get the identifier of the book to display
 String bookId = request.getParameter("bookId");
 if (bookId != null) {
 // and the information about the book and print it
 ...
 }
 out.println("</body></html>");
 out.close();
 }
 ...
}
```

- The servlet extends the `HttpServlet` class and overrides the `doGet` method.
- Within the `doGet` method, the `getParameter` method gets the servlet's expected argument.
- To respond to the client, the example `doGet` method uses a `Writer` from the `HttpServletResponse` object to return text data to the client. Before accessing the writer, the example sets the content-type header. At the end of the `doGet` method, after the response has been sent, the `Writer` is closed.

Handling POST requests involves overriding the `doPost` method. The following example shows the `ReceiptServlet` doing this.

```
public class ReceiptServlet extends HttpServlet {

 public void doPost(HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException
 {
 ...
 // set content type header before accessing the Writer
 response.setContentType("text/html");
 PrintWriter out = response.getWriter();

 // then write the response
 out.println("<html>" +
 "<head><title> Receipt </title>" +
 ...);

 out.println("<h3>Thank you for purchasing your books from us " +
 request.getParameter("cardname") +
 ...);
 out.close();
 }
 ...
}
```

- The servlet extends the `HttpServlet` class and overrides the `doPost` method.
- Within the `doPost` method, the `getParameter` method gets the servlet's expected argument.
- To respond to the client, the example `doPost` method uses a `Writer` from the `HttpServletResponse` object to return text data to the client. Before accessing the writer the example sets the content-type header. At the end of the `doPost` method, after the response has been set, the `Writer` is closed.

# SendRedirect() and RequestDispatcher() in Servlet

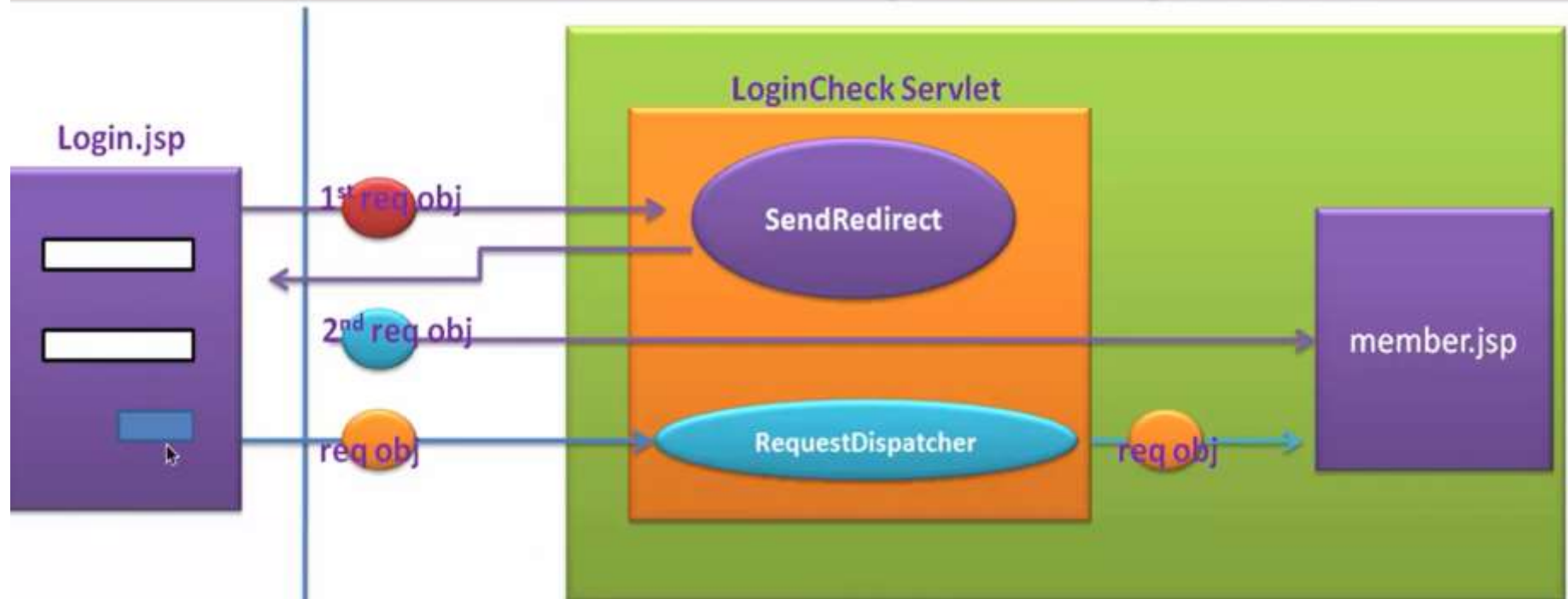
## SendRedirect

- This is the method of object **HttpServletResponse**.
- Request is redirected to client (Browser), and it will process the new URL.
- End User can see on which page, url is redirected.
- In Nutshell, **Processing done at client side**.

## RequestDispatcher

- This object can be accessed from **HttpServletRequest**.
- Servlet will internally forward the request to another servlet or jsp page.
- End user don't know that which page is processed internally.
- In Nutshell, **Processing done at server side**.

# SendRedirect and RequestDispatcher



## Demerits of SendRedirect

- 1) Round trip through the browser decreases performance
- 2) The first request object containing important information is lost

In RequestDispatcher these problems are solved

When to use SendRedirect?

If we want to go to a page out side of the application

# Database Access with Servlets

- **Import JDBC and Servlet Classes**
- **Define Database Configuration**

```
String jdbcURL = "jdbc:mysql://localhost:3306/your_database";
```

```
String jdbcUsername = "root";
```

```
String jdbcPassword = "your_password";
```

- **Load the JDBC Driver (Optional for modern JDBC)**

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- **Create a Connection:-**

```
Connection conn = DriverManager.getConnection(jdbcURL, jdbcUsername,
jdbcPassword);
```

- **Prepare SQL Statement**
- **Execute Query**
- **Close Resources**

# JavaBean

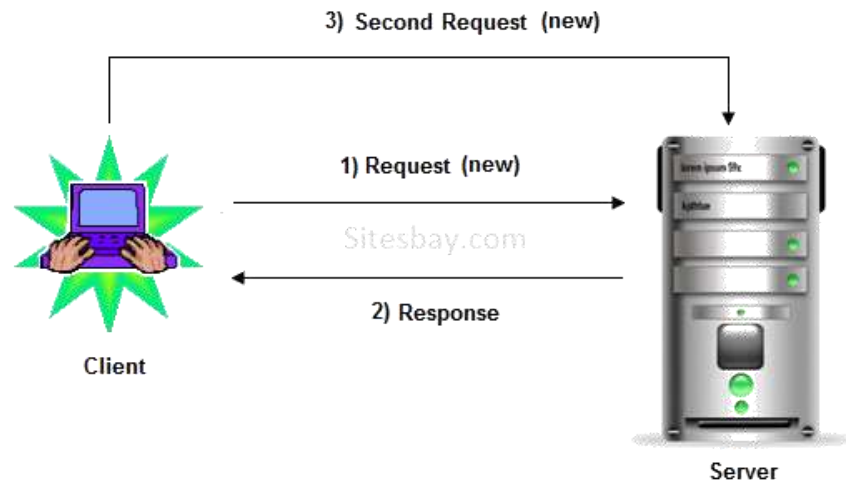
- Useable software component that encapsulates data and functionality in a java class.
- Java bean is a java class that is developed by following some standards.
- Java bean does-not contain any logic.
- It acts as helper class to carry data from one class to another or from one project to another.
- In order to pass more than 3 values from one class to another or from one project to another, take support of java beans.

The standards to develop a java bean class are following:-

- 1) Class must be taken as public :-So it is accessible from other classes and frameworks.
- 2) Must have a public default (no-argument) constructor:-Allows easy instantiation using reflection.
- 3) Properties should be private:-Use **encapsulation** to protect data.
- 4) Provide public getter and setter methods:-For accessing and modifying the private properties.
- 5) Implements Serializable interface (optional but recommended):-So the bean can be saved, sent over networks, or stored in sessions.

# Handling Session.

- HTTP is a stateless protocol.
- Each request is independent of the previous one.
- However, in some applications, it is necessary to save state information so that information can be collected from several interactions between a browser and a server.
- Sessions provide such a mechanism.
- A session can be created via the `getSession( )` method of `HttpServletRequest`.
- An `HttpSession` object is returned.
- This object can store a set of bindings that associate names with objects. The `setAttribute( )`, `getAttribute( )`, `getAttributeNames( )`, and `removeAttribute( )` methods of `HttpSession` manage these bindings.
- It is important to note that session state is shared among all the servlets that are associated with a particular client.



```
//index.jsp
<html>
 <head>
 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
 <title>Testing Cookies</title>
 </head>
 <body>
 <label style="color: blue">Testing Session</label></br>
 <label>Click below to get Session Value</label></br>
 click here
 </body>
</html>
```

```
//getSession.java
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Date;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

protected void processRequest(HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException {
 response.setContentType("text/html;charset=UTF-8");
 PrintWriter out = response.getWriter();
 try {
 // Get the HttpSession object.
 HttpSession hs = request.getSession(true);
 // Get writer.
 // response.setContentType("text/html");
 }
}
```

```
//PrintWriter pw = response.getWriter();
out.print("");
// Display date/time of last access.
Date date = (Date)hs.getAttribute("date");

// Display current date/time.

out.println("<html>");
out.println("<head>");
out.println("<title>Servlet GetSession</title>");
out.println("</head>");
out.println("<body>");
if(date != null) {
 out.print("Last access: " + date + "
");
}
date = new Date();
hs.setAttribute("date", date);
out.println("Current date: " + date);
out.println("</body>");
out.println("</html>");
} finally {
 out.close();
}
}
```

# Handling cookie.

- Cookie are the textual information which are stored in key value pair format to the client's browser during multiple requests.
- In order to handle cookie in java, there is a cookie class in java present in `javax.servlet.http` package.
- To make cookie just create a object of cookie class and pass name and its value.
- To add cookie in response just use `addCookie(cookie)` method of response interface.

# Questions

- Write a java program using servlet to display "Tribhuvan University".
- What is java Bean? How is it different from other java classes? Discuss property design patterns with suitable of each.
- Explain the life cycle of a servlet.
- How forms can be created and processed using JSP? Make it clear with your own assumptions.
- Describe the process to deploy servlet. Write a Program to create a JSP web form to take input of a student and submit it to second JSP file which may simply print the values of form submission.
- Explain the significance of cookies and sessions with suitable example.
- Create a login form in login.jsp, using Servlet authenticate (accept **POST** request) the username (as **admin**) and password (as **nccs#123**), forward to dashboard.jsp if login is successful and include error message if login fails in same login.jsp. [10]
- Explain JSP Architecture.
- Write a simple JSP program to display "Kathmandu, Nepal" 10 times.
- Discuss MVC design pattern with example.